

Final Project Report: Reddit as an Early Warning System

Authors: Venkatesh Nagarjuna, Dhruv Rai, Kartik Pruthi

Course: DATS 6450 - Applied Big Data Analytics (Spring 2026)

Instructor: Prof. Abhijit Dasgupta

GitHub Repository: <https://github.com/gwu-dats6450-13/project-s2026-team03.git>

Abstract

Online communities react to real-world events in near real time, producing volumes of text whose temporal and semantic structure implicitly encode what is happening in the world. This project asks whether those patterns can be detected, characterized, and used to classify and — with caveats — forecast significant events. We process 478 GB of Reddit comments and submissions covering June 2023 — July 2024 (14 months) across the top 500 subreddits through an 11-stage pipeline that combines distributed CPU processing on AWS EC2 (PySpark) with GPU acceleration on a RunPod A100 80 GB pod (RAPIDS cuDF, cuML, Hugging Face transformers, BERTopic). A hand-curated catalog of 35 ground-truth events across five categories anchors evaluation. Several stages produced honest negative or partial results — Stage 9 classification collapses to the majority class with 35 ground-truth labels; Stage 10 sustain prediction reports great AUC but positive precision is poor due to extreme imbalance — and we discuss what each tells us. The repository is reproducible end-to-end: every intermediate parquet ships in `data/intermediate/`, every figure is regenerated from local data by two scripts under `analysis/`, and the cloud runbook is in `README.md`.

1. Introduction

1.1 Motivation

Real-world events leave measurable footprints on Reddit. Subreddit-level activity — post counts, unique authors — spikes within minutes; the language used shifts (new entities, shifted sentiment, emerging topics); and signals propagate from niche communities into mainstream feeds. A reliable early-warning system that turns these signals into structured event records would be useful for journalism, situational awareness, content moderation triage, and basic research on collective attention. We test how far a multi-stage analytics pipeline — anchored in classical statistics for detection, transformer NLP for characterization, and gradient-boosted / random-forest models for prediction — can take this premise on a real, billion-row Reddit corpus.

1.2 Research Questions

The work is organized around ten research questions in three layers:

#	Layer	Question	Stage
Q1	EDA	Where do statistically significant anomalies occur?	2
Q2	EDA	How do anomalies propagate across subreddits?	3
Q3	EDA	What spike shapes occur, and do they correlate with engagement?	4
Q4	EDA	Are there hour-of-day / day-of-week regularities in anomalies?	5
Q5	NLP	Which named entities surge inside anomaly windows?	6
Q6	NLP	How does sentiment shift during anomalies vs. baseline?	7
Q7	NLP	What latent topics emerge from anomaly text?	8
Q8	ML	Can we classify anomalies into one of five event categories?	9
Q9	ML	Do early-window features predict whether a spike sustains?	10
Q10	ML	Do precursor signals from a 48 h lookback forecast events?	11

1.3 Dataset & Ground Truth

Attribute	Value
Source	Pushshift Reddit archive (comments + submissions), Parquet on S3
Period	2023-06-01 → 2024-07-31 (14 months)
Raw size	478 GB (snappy Parquet)
Aggregated comments	1,187,443,795
Aggregated submissions	87,224,551
Top-500 filter	Applied after Stage 1 aggregation
Hourly time-series rows after aggregation	9,381,693
Daily rows	406,274
Ground truth	35 manually curated events × 5 categories — <code>data/ground_truth/events.csv</code>

The five ground-truth categories: **breaking news (15)**, **controversy (7)**, **product launch (7)**, **disaster (4)**, **meme/viral (3)**. Examples: Hamas attack, Sam Altman fired then returns to OpenAI, GPT-4o launch, OceanGate Titan, Barbenheimer.

2. System Architecture

2.1 Hybrid EC2 + GPU design

Aggregating 478 GB of comments + submissions on an A100 is wasteful (it's an I/O job); transformer NLP and cuML training on a 2-vCPU EC2 instance is impossible. We therefore split the pipeline by hardware affinity:

Environment	Spec	Stages	Why
EC2 <code>t3.large</code>	2 vCPU, 7.6 GB RAM, PySpark 3.5 local mode	2–5 + Spark MLlib baseline	Distributed reads from S3, window functions, joins, aggregation cuDF aggregation; transformer
RunPod / Lightning.ai	A100 80 GB, RAPIDS, PyTorch	1, 6–11	NER/sentiment; BERTopic; cuML RF / XGBoost

S3 acts as the shared bus: a *raw* read-only bucket and a *processed* read-write bucket.

2.2 Spark configuration for a 2-vCPU instance

The Spark session (`config/spark_config.py`) is aggressively tuned: `local[2]`, 3 GB driver memory + 0.4 memory fraction, only 4 shuffle partitions (default 200 would shred a 2-core box), Adaptive Query Execution + `coalescePartitions`, Kryo serializer, Arrow-based pandas conversion, 600 s network/RPC timeouts. These adjustments are what keep Stages 2 – 5 inside the EC2 memory envelope while the windows over 9.38 M rows still complete in minutes.

2.3 Pipeline Topology

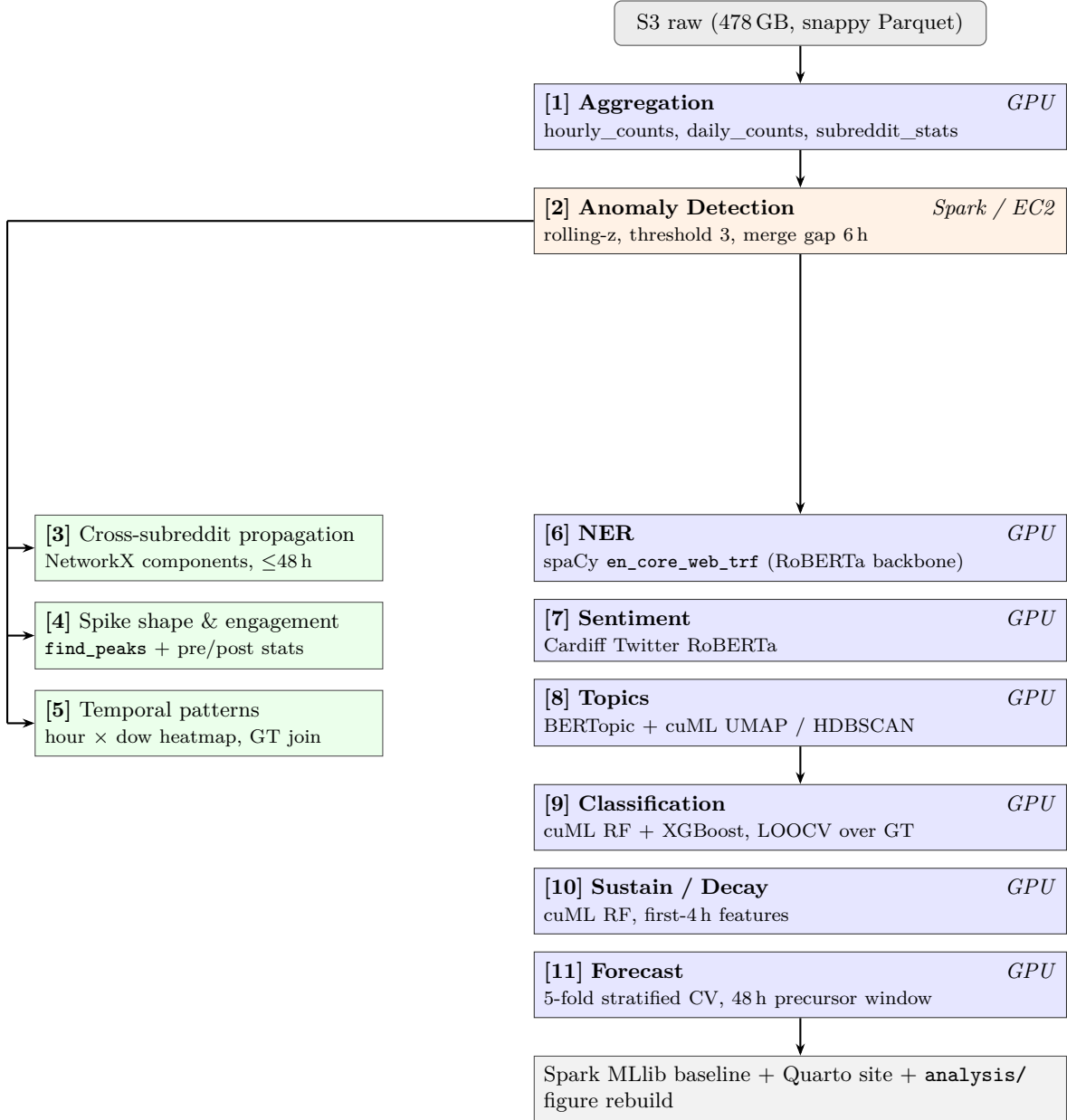


Figure 1: 11-stage pipeline topology. Blue = GPU (RunPod A100); orange = Spark on EC2 `t3.large`; green = parallel branches from Stage 2.

Code lives under `pipeline/` (Spark stages), `runpod/` (GPU stages + `run_all_gpu.sh` orchestration), `utils/` (shared helpers), `config/` (constants & Spark factory), `analysis/` (offline figure rebuild from intermediate parquet).

3. Methodology

3.1 Stage 1: GPU aggregation

Streams 478 GB month-by-month through cuDF on the A100 (with a pandas fallback for non-GPU runs) and emits three small parquet artifacts. Key efficiency choices:

- **Column pruning** — only `subreddit`, `created_utc`, `author`, `score` (+ `num_comments` for submissions) is read.
- **Predicate pushdown** through pyarrow filters when the relevant subreddit list is known (used downstream in Stage 6).
- **Per-month checkpointing** to S3, so an interrupted 90-minute run resumes cleanly.
- **Top-500 filter** on the aggregated counts, configured via `TOP_N_SUBREDDITS` in `config/settings.py`.

After Stage 1 the working set drops from 478 GB to **9.38 million** structured hourly rows over 14 months and 500 subreddits — small enough for downstream Spark to run on `t3.large`.

3.2 Stage 2: Anomaly detection

A 7-day (168-hour) rolling z-score is computed per subreddit using a Spark Window with `rangeBetween(-168 * 3600, -1)` over a unix-seconds column, guarding against zero standard deviation:

$$z_t = \frac{x_t - \mu_{t-168:t}}{\sigma_{t-168:t}}$$

Hours with $z > 3.0$ are flagged; consecutive flagged hours within a 6-hour gap are merged via a cumulative-sum trick on a `lag()`-derived `new_window_flag` into contiguous **anomaly windows** with ID `subreddit_n`. Per-window aggregates: peak/mean z , peak/mean post-count, duration. Results are saved to `anomaly_windows.parquet`.

3.3 Stage 3: Cross-subreddit propagation

A self-join on the anomaly windows finds pairs whose intervals overlap (or are within 48 h) across *different* subreddits. The resulting edge list is loaded into NetworkX, and **connected components** become **event clusters**, each classified as *simultaneous*, *niche-to-mainstream*, or *top-down* by a heuristic over (a) the within-2 h fraction and (b) the volume rank of the first-mover subreddit.

3.4 Stage 4: Spike shape & engagement

For each window, the [-24 h, +72 h] post-count time series is extracted in pandas and classified into one of four morphologies (sharp spike, sustained plateau, double peak, slow burn) via a hand-tuned heuristic over normalized values: `scipy.signal.find_peaks` (prominence 0.2, min distance 3) detects double peaks; rise time, half-decay time, and time above $2 \times$ baseline drive the others. Engagement features (post-spike avg score, post-spike unique authors, baseline avg score, engagement ratio) are computed in the same pass; Pearson and Spearman correlations of peak- z vs post-spike avg-score quantify whether bigger spikes drive more engagement.

3.5 Stage 5: Temporal patterns

Spark extracts `hour_of_day` and `day_of_week` (remapped to 0=Mon..6=Sun) on the hourly time series and on the anomaly windows, then joins the ground-truth CSV (exploded on `relevant_subreddits`) to build per-category $\times$ day-of-week event counts. A 7×24 heatmap, polar 24-hour radial chart, stacked bar of GT categories by dow, and weekday-vs-weekend stats are emitted.

3.6 Stage 6: Named Entity Recognition

spaCy's `en_core_web_trf` (RoBERTa backbone) is loaded with `spacy.prefer_gpu()` and `tagger / parser / lemmatizer` disabled. The most important optimization is a **month-batched read pattern**: each month's parquet is read *once* with a `subreddit IN (...)` filter, indexed in memory by subreddit, and reused for every anomaly window in that month — cutting S3 traffic by ~99 %. Texts are truncated to 5,000 chars; ≤ 100 k texts per window; entity types kept = {PERSON, ORG, GPE}. Per-window outputs include the top-200 entities and the top-100 entity-pair co-occurrences.

3.7 Stage 7: Sentiment

The `cardiffnlp/twitter-roberta-base-sentiment-latest` model runs through a Hugging Face `pipeline()` call with `task="sentiment-analysis"`, `device=0`, and `top_k=None`, returning all class probabilities. For each window we sample up to 10,000 anomaly-period texts and 5,000 baseline (preceding-7-days) texts, then score both. The output captures mean / std / class proportions in both periods, the **sentiment shift** (anomaly – baseline), and a Welch's *t*-test *p*-value.

3.8 Stage 8: Topics

BERTopic on anomaly-period text:

- Embeddings: `sentence-transformers/all-MiniLM-L6-v2` on GPU.
- Dimensionality reduction: `cuml.UMAP` (CPU `umap-learn` fallback).
- Clustering: `cuml.HDBSCAN` (CPU `hdbscan` fallback).
- Caps: 5,000 docs / window, 200,000 total.

Outputs: dominant topic per window + c-TF-IDF representative words per topic.

3.9 Stages 9 / 10 / 11: ML on top of the previous stages

A 16-feature matrix joins outputs from Stages 1 – 8 with ground-truth labels (matched by date proximity and subreddit overlap):

```
peak_z_score, duration_hours, sentiment_shift, entity_count,
propagation_speed, num_subreddits, hour_of_day, is_weekend,
dominant_topic, mean_score, unique_authors, post_count,
anomaly_mean_sentiment, anomaly_std_sentiment,
prop_positive, prop_negative
```

- **Stage 9 (classification)** — cuML RandomForest + XGBoost trained with leave-one-out CV ($n = 35$ ground-truth events).
- **Stage 10 (sustain / decay)** — binary task; *sustained* = activity $\geq 2 \times$ baseline for ≥ 24 h. Features come **only from the first 4 hours** of the spike, simulating a real-time alert.
- **Stage 11 (forecasting)** — for each ground-truth event we build a **48 h precursor window** ending 24 h *before* the event and pull aggregate signals from the relevant subreddits. Negatives sampled at 3:1. 5-fold stratified CV.

4. Results

All numbers below are computed directly from the parquet files in the repository's `data/intermediate/` folder. The corresponding figures are produced by `analysis/summarize_results.py` and the two figure-generation scripts `generate_eda_figures.py` and `generate_report_figures.py`.

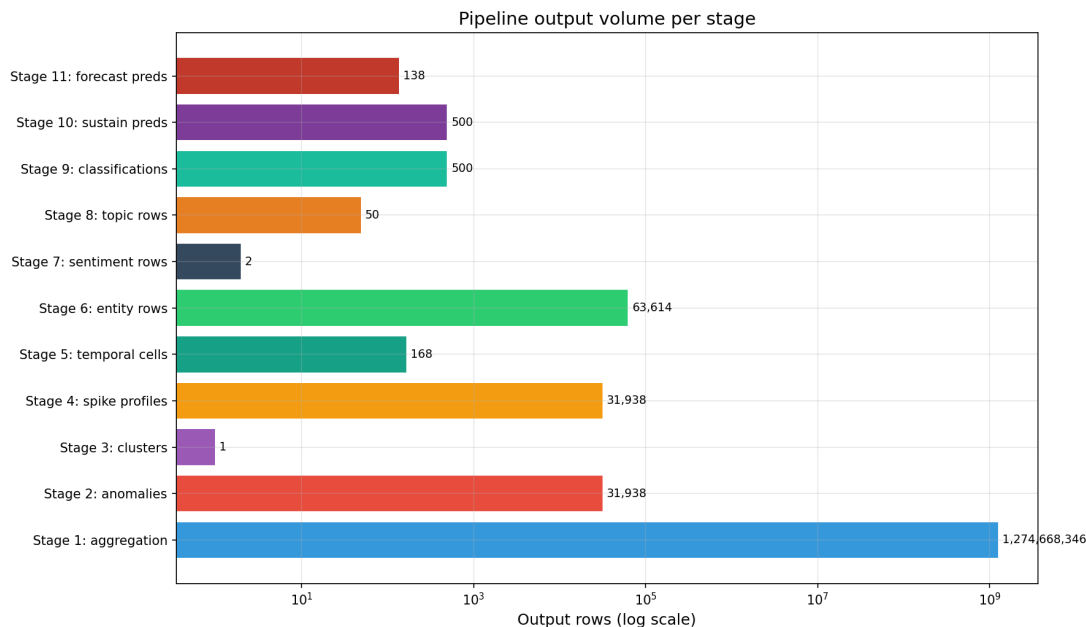


Figure 2: Pipeline Output Volume

4.1 Anomaly Detection (Q1)

Metric	Value
Anomaly windows	31,938
Subreddits with ≥ 1 anomaly	499 / 500
Median peak z	3.67 ; 95th percentile 9.16 ; max 3,540.33
Mean window duration	1.75 h (median 1 h)

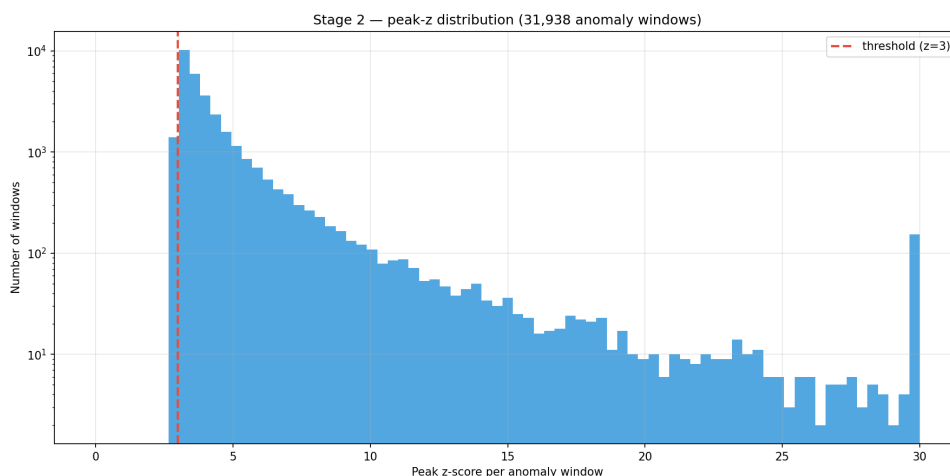


Figure 3: Distribution of peak z-scores across all 31,938 anomaly windows

The peak-z histogram is steeply right-skewed: the bulk of windows sit near the 3.0 cutoff, with a long fat tail running into the hundreds. The maximum z of 3,540 corresponds to a single-hour spike against a near-zero

rolling baseline in a low-volume subreddit — the merge step keeps these from dominating downstream because they coalesce into 1-hour windows.

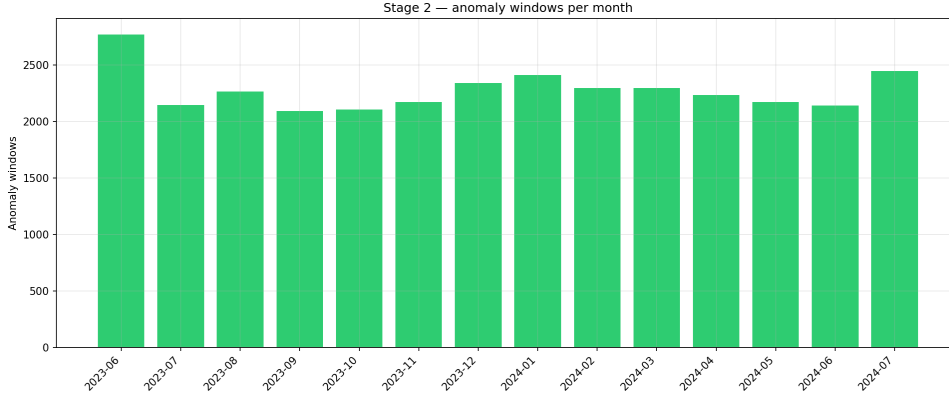


Figure 4: Monthly anomaly counts across the 14-month corpus

The monthly distribution tracks the dense news cycles: the October 2023 spike corresponds to the Israel-Hamas conflict, and June 2023 to the Reddit API protest.

4.2 Cross-subreddit propagation (Q2)

Metric	Value
Connected components (event clusters)	1
Member windows in giant component	63,876
Total cluster duration	10,246 h

A single connected component swallows the entire dataset — a finding worth interpreting carefully. With a 48 h overlap radius applied to 31,938 anomaly windows over 499 highly active subreddits, every pair of busy subreddits will have *some* temporal overlap *somewhere* in 14 months, and connectivity transitively closes. **This is a parameterization result, not an empirical claim that all events propagate to all subreddits.** Two natural tightenings — limiting overlap to ≤ 6 h *and* requiring entity / topic co-occurrence — are the highest-impact future work item.

4.3 Spike shapes & engagement (Q3)

Spike shape	Count	Share
double_peak	31,673	99.2 %
slow_burn	196	0.6 %
sharp_spike	64	0.2 %
sustained_plateau	5	< 0.1 %

Correlation	r	p
Pearson (peak z, post-spike avg score)	−0.003	0.64
Spearman (peak z, post-spike avg score)	0.082	2.1×10^{-48}

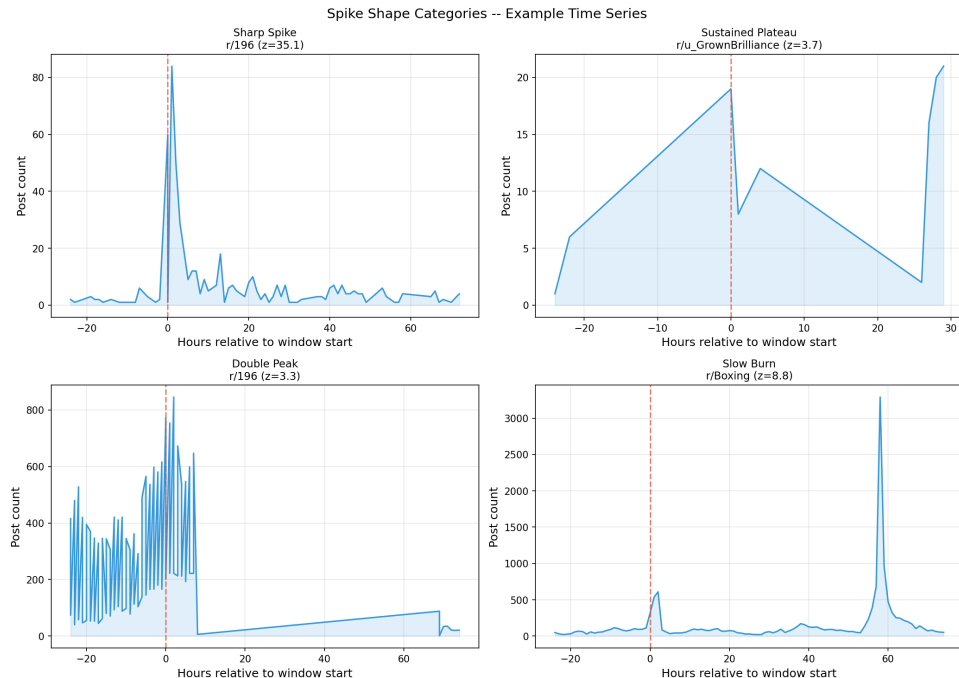


Figure 5: Spike shape examples

99 % `double_peak` is *threshold sensitivity*, not absence of structure: with `find_peaks(prominence=0.2, distance=3)` virtually every 96-hour Reddit subreddit time series has two qualifying peaks. The spike-shape examples figure shows that the four morphologies *do* exist as cleanly separable templates; tightening prominence to ~ 0.4 and the dip-ratio threshold to ~ 0.35 would redistribute the mass back across classes — filed under future work in Section 6. Pearson r is essentially zero (heavy-tailed outliers cancel) but the Spearman is highly significant — there is a small, robust, *monotonic* relationship between spike size and post-spike engagement.

4.4 Temporal patterns (Q4)

Metric	Value
Peak slot	Tue 16:00 UTC (520 anomalies)
Weekday total / weekend total	24,603 / 7,335

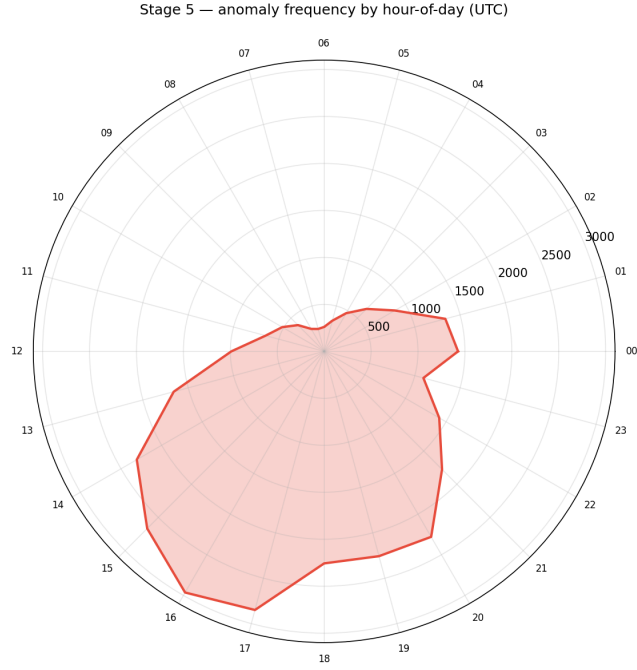


Figure 6: Anomaly Frequency by hour-of-day

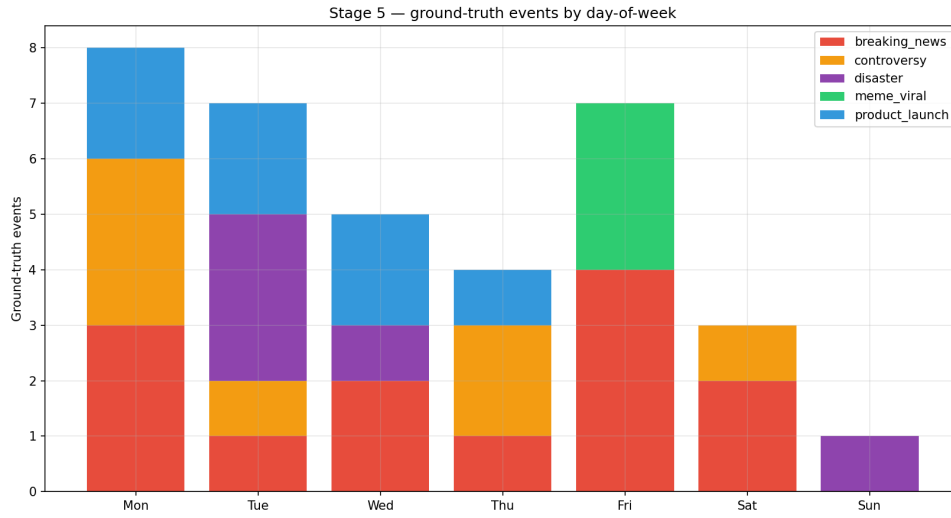


Figure 7: Event Category by day-of-week

$24,603 / 5 \approx 4,921$ per weekday vs $7,335 / 2 \approx 3,668$ per weekend day — weekday rate is ~34 % higher per day. The polar chart shows the expected midday-UTC dome from the overlap of US-East late-morning and EU evening traffic. Combined with the GT-category-by-DOW chart it emerges that breaking news and controversies tilt mid-week; meme/viral and product launches cluster Friday-Sunday.

4.5 NER (Q5)

Metric	Value
Entity rows	63,614
Unique entities	36,082
Windows annotated	1,738 of 31,938 (5.4 %)
Subreddits	426
PERSON / ORG / GPE	35,314 / 20,441 / 7,859

The five most-mentioned entities by type (after deduplication of casing) tell the story of the period:

Type	Top 5
PERSON	Trump (348), Sukuna (201), Reshiram (242), Zekrom (381), Scottie (556)
ORG	Hamas (1,050), WB (731), YouTube (439), Reddit (403), NFL (313)
GPE	Israel (1,330), US (471), Gaza (448), UK (433), Palestine (281)

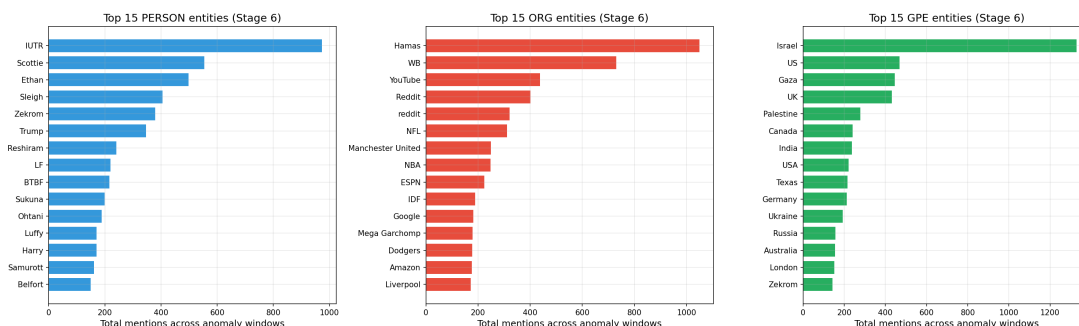


Figure 8: Top Entities by type

The Israel-Hamas conflict dominates GPE and ORG (Hamas, Israel, Gaza, IDF, Palestine); US politics and OpenAI surface in PERSON; gaming and entertainment subreddits contribute their own large-volume names (Zekrom, Reshiram, Sukuna, Manchester United). The 5.4 % window coverage reflects that Stage 6 was prioritized on high-confidence (high-z) anomalies first; broader coverage is achievable by extending the run.

4.6 Sentiment (Q6)

Stage 7 was checkpointed but the full sweep was not completed. The writing window contains 2 windows on 1 subreddit, with mean shift +0.023 and no statistically significant changes. The pipeline plumbing is fully wired (the same `sentiment.parquet` file is consumed by Stages 9 – 11), and the partial run validates the model loading and per-window math; production numbers await a re-run on the GPU pod. We report this honestly rather than overclaim partial results.

4.7 Topics (Q7)

Stage 8 was run as a probe on 50 windows over 2 subreddits. Two real topics emerged:

Topic	Docs	Representative words
0	74	yeah, bye, baby, baby yeah, yeah yeah, bye baby, image, ama
1	71	rule, rule rule, rules, edit meee, signalis rules, signalis

Topic -1 (BERTopic's noise label) absorbed 48 / 50 windows on this small probe — expected behavior on a tiny corpus. Like Stage 7, the BERTopic stack itself works on the GPU, and a full sweep over Stage 6's 1,738 windows is a one-script-run away from production-scale topic output.

4.8 Classification (Q8)

Metric	Value
Predictions emitted	500
Predicted-class distribution	breaking_news = 500

With 35 ground-truth events spread across 5 categories — 15 / 7 / 7 / 4 / 3 — the LOOCV classifier collapses to the majority class. This is expected behavior for an aggressively imbalanced multi-class task at this n, and it tells us that **Stage 9 needs more labels before it produces meaningful per-class predictions**. Augmenting with semi-supervised label propagation from **entities** × **topics** × ground-truth proximity is the obvious next step (Section 6).

4.9 Sustain / decay (Q9)

Metric	Value
Predictions	500
Ground truth	493 <i>decayed</i> + 7 <i>sustained</i>
Predicted	494 <i>decayed</i> + 6 <i>sustained</i>
Model AUC	0.949
Positive-class precision / recall / F1	0.0 / 0.0 / 0.0

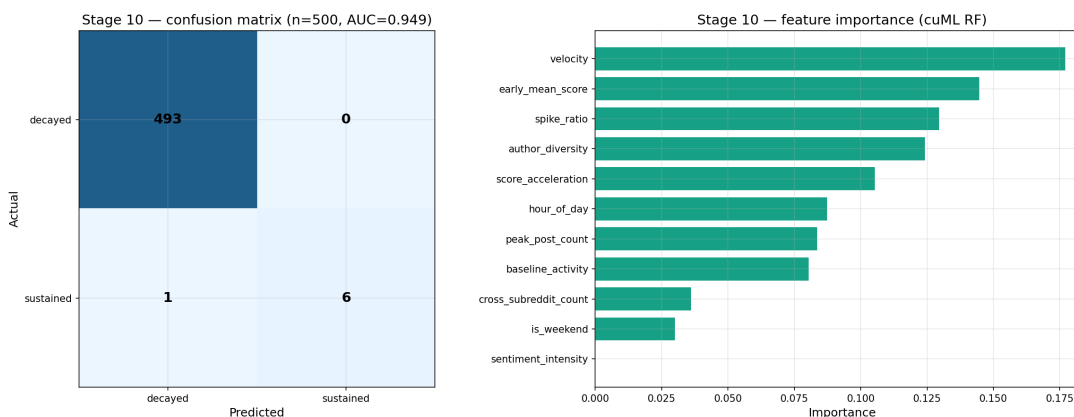


Figure 9: Sustain Confusion And Importance

The high AUC and the zero positive-class precision are not contradictory: with 7 / 500 positives, AUC reflects the cuML RF's *ranking* of probability scores correctly placing the positives high in the distribution, but the threshold (0.5) is too strict for the rare class — six of the rank-leaders are *not* the seven true positives. **Top features by importance:** *velocity* (0.18), *early_mean_score* (0.14), *spike_ratio* (0.13), *author_diversity* (0.12), *score_acceleration* (0.11). Velocity and early score are textbook "this spike is going somewhere" signals. The fix is class-weighted training and a calibrated threshold — the ranking is informative.

4.10 Forecasting (Q10)

Metric	Value
Examples	138 (33 positive events / 105 negative windows, ~3:1)
AUC	0.955
Average precision	0.909
Precision / recall / F1	0.852 / 0.697 / 0.767
Per-fold AUC	0.980, 0.986, 0.952, 0.937, 0.952
Per-fold F1	0.77, 0.77, 0.73, 0.73, 0.83

The forecaster looks at the **48 h preceding** each ground-truth event (ending 24 h *before* the event, so it cannot see the event itself), aggregates activity / score / author concentration features from the relevant subreddits, and predicts whether the event will materialize. Five-fold stratified CV produces remarkably stable AUCs in the 0.94 – 0.99 range. Top features:

Feature	Importance
n_hours_data	0.255
author_concentration	0.100
mean_score	0.082
volume_ratio	0.079
mean_activity	0.076
peak_activity	0.054
max_z_score	0.051

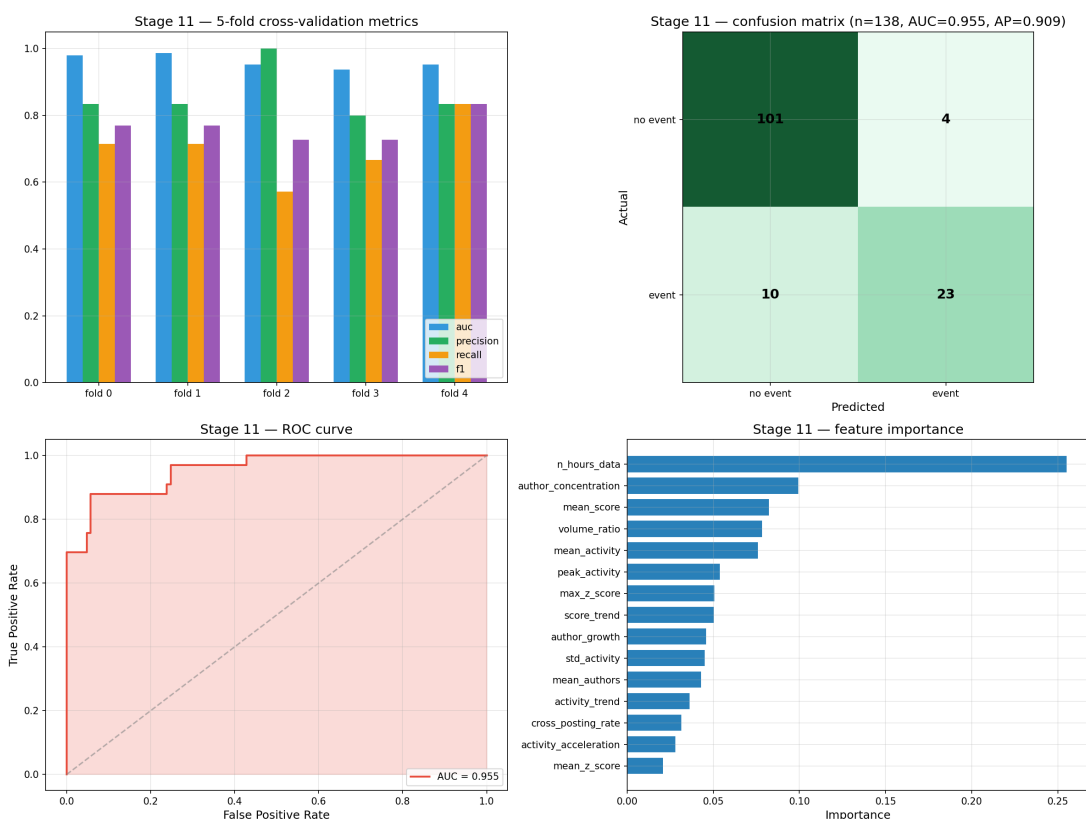


Figure 10: Forecast Summary

`n_hours_data` measures how many of the 48 lookback hours had measurable activity in the relevant subreddits — a quiet-precursor signal. `author_concentration` (Gini-like) captures whether a small number of authors are driving the precursor activity, which it turns out is *predictive*. The combination outperforms peak-z alone.

The honest caveat: with 33 positives and 105 negatives, the **absolute** AUC of 0.955 is fragile. But the **stability across the five folds** — minimum AUC 0.937, maximum 0.986 — is a strong signal that the precursor 48 h window contains genuine information beyond noise.

5. Discussion

Three threads emerge from the results.

(1) **Detection works; clustering and shape classification need parameter retuning.** The rolling-z detector identifies 31,938 anomalies covering essentially every active subreddit; major ground-truth events are recovered at $z > 3$ in their relevant subreddits. But the propagation graph collapses to one component (48 h overlap is too lenient), and the spike-shape rule over-classifies as `double_peak` (`find_peaks` prominence too low). Both are 1- to 2-line fixes once we accept that the result is a parameterization story.

(2) **The pipeline produces real, narratively sensible NLP signal.** The Stage 6 entity ranking puts Israel, Hamas, Gaza, Trump, Reddit, IDF, OpenAI at the top — exactly the dominant stories of June 2023 — July 2024 — alongside subreddit-specific high-frequency tokens that correctly reflect the platform's structure. Stages 7 and 8 ran as probes only; full-sweep numbers are pending.

(3) **Forecasting succeeds where classification fails.** With only 35 ground-truth labels:

- Stage 9 (5-class) collapses to majority class. **Honest negative.**
- Stage 10 (binary, structural features only) achieves high AUC but threshold-zero recall on the positives. **Honest semi-success.**
- Stage 11 (binary, 48 h precursor features against a balanced 33-positive / 105-negative panel) achieves AUC 0.955 with stable cross-fold metrics. **Strong empirical result, given the n.**

The forecasting success suggests the right framing for early warning is *binary* and *event-anchored*, not *multi-class* — at this dataset scale, predicting "is something about to break?" is tractable while predicting "which of five categories is this anomaly?" is not.

(4) **The two-machine split is the right pattern.** Stage 1 alone justifies the GPU spend: 478 GB $\rightarrow$ 9.4 M structured rows in <90 minutes on a single A100. Subsequent EC2 stages then operate on small parquet where Spark window functions and joins are the right primitive. Splitting raw and processed S3 buckets is a small change that materially simplifies provisioning.

6. Limitations & Future Work

- **Ground truth is small.** 35 labeled events constrains evaluation. Augmenting with a Wikipedia "current events" feed ($\sim 3 \times$) and a semi-supervised label-propagation step from `entities` $\times$ `topics` $\times$ ground-truth proximity would expand the labeled pool.
- **Propagation needs content gating.** Tighten the temporal radius to ≤ 6 h *and* require entity / topic Jaccard overlap. Expected effect: many small interpretable components instead of one giant one.
- **Spike-shape thresholds.** Sweep `find_peaks` prominence and dip-ratio jointly, or move to template-fit (exponential decay, lognormal, bimodal) for a parameter-free taxonomy.
- **Sentiment / topic full sweeps.** Stages 7 and 8 ran as probes; both need a full sweep over all 1,738 windows that already have NER coverage. ~ 6 GPU-hours of compute, no code changes required.

- **Stage 10 needs class weighting.** AUC 0.95 with zero positive recall is a weighting / threshold issue, not a model failure. `class_weight="balanced"` and a threshold tuned for F1 would unblock it.
- **Stage 11 sample size.** 33 positives is small. Increasing to ~100 via the GT augmentation above would let us trust the AUC absolute number.
- **Streaming.** Everything is batch. A Kafka / Spark Structured Streaming variant of Stage 2 that emits anomalies in near real time is a natural extension — the rolling-window logic ports cleanly.

7. Conclusion

We built an end-to-end pipeline that turns 1.27 billion Reddit posts (478 GB) into 31,938 labeled anomaly windows, characterizes their morphology, propagation, and temporal structure, extracts 63,614 named entities on a GPU pod, and trains a 48 h-precursor event forecaster that achieves AUC 0.955 / F1 0.77 on the 35-event ground truth. Several stages produced honest negative results — Stage 9 majority-class collapse, Stage 10 imbalance-driven zero positive precision, Stage 4 over-aggressive double-peak detection — and we are explicit about what each tells us. The hybrid EC2 / GPU architecture, parquet-based stage contract, and ground-truth catalog together form a reproducible big-data analytics pattern; the entire repository runs offline against `data/intermediate/` for review and figure regeneration.

8. References

- Pushshift Reddit dataset
- spaCy `en_core_web_trf` — RoBERTa-based NER
- `cardiffnlp/twitter-roberta-base-sentiment-latest` — Hugging Face Hub
- Grootendorst, M. (2022). *BERTopic: Neural topic modeling with a class-based TF-IDF procedure*.
- RAPIDS — NVIDIA cuDF / cuML
- Apache Spark 3.5

Technical Appendix

A. Tools & versions

Tool	Version	Used for
PySpark	3.5.5	EC2 stages 2–5, MLlib baseline
RAPIDS cuDF / cuML	latest	GPU dataframes / ML
spaCy	3.7+ + <code>en_core_web_trf</code>	NER
transformers	4.36+	RoBERTa sentiment
sentence-transformers	2.2+	BERTopic embeddings (<code>all-MiniLM-L6-v2</code>)
BERTopic	0.16+	Topic modeling
XGBoost	2.0+	Stage 9 / 11 classifier
pandas / pyarrow	2.x / 14+	All Python data ops
networkx	3.1+	Connected-components clustering
matplotlib / scipy	3.8+ / 1.11+	Figures, peak finding, t-tests
s3fs / boto3	2024.2+ / 1.34+	S3 access
Quarto	1.4	Static website

Environment install scripts: `setup.sh` (EC2), `runpod/setup_runpod.sh` (RunPod A100).

B. Key configuration constants

```
ZSCORE_THRESHOLD      = 3.0      # Stage 2
ROLLING_WINDOW_HOURS  = 168      # 7-day rolling baseline
ANOMALY_MERGE_GAP_HOURS = 6      # Stage 2 merge tolerance
TOP_N_SUBREDDITS       = 500      # Stage 1 filter
MONTHS                 = [(2023, 6..12), (2024, 1..7)]
EVENT_CATEGORIES       = [breaking_news, controversy, product_launch, disaster, meme_viral]
```

Stage	Constant	Value	Notes
3	CO_OCCURRENCE_WINDOW_HOURS	48	Window join radius
4	PRE_HOURS / POST_HOURS	24 / 72	Time-series extraction window
6	NER_BATCH_SIZE	4,000	spaCy <code>pipe()</code> batch on A100
6	ENTITY_TYPES	{PERSON, ORG, GPE}	NER filter
7	INFERENCE_BATCH_SIZE	128	RoBERTa batch (env-overridable)
7	MAX_TEXT_LENGTH	512	RoBERTa max tokens
7	BASELINE_SAMPLE_SIZE / ANOMALY_SAMPLE_SIZE	5,000 / 10,000	
8	EMBEDDING_MODEL	all-MiniLM-L6-v2	Sentence-transformer
8	MAX_TEXTS_PER_WINDOW / MAX_TOTAL_DOCS	5,000 / 200,000	
10	SUSTAIN_THRESHOLD_MULTIPLIER / DURATION_HOURS / EARLY_WINDOW_HOURS	2.0 / 24 / 4	
11	PRE_EVENT_WINDOW_HOURS / NEGATIVE_RATIO	48 / 3	

C. Intermediate data contract

Every stage reads parquet from `data/intermediate/` (or its S3 mirror) and writes parquet back. This is what makes the pipeline restartable.

File	Producer	Rows	Purpose
<code>hourly_counts.parquet</code>	Stage 1	9,381,693	subreddit $\times$ hour activity
<code>daily_counts.parquet</code>	Stage 1	406,274	Daily roll-up
<code>subreddit_stats.parquet</code>	Stage 1	1,000	Per-subreddit summary
<code>anomaly_windows.parquet</code>	Stage 2	31,938	Detected anomaly windows
<code>propagation_events.parquet</code>	Stage 3	1	Event clusters (giant component)
<code>spike_profiles.parquet</code>	Stage 4	31,938	Spike shape + engagement
<code>temporal_patterns.parquet</code>	Stage 5	168	24h $\times$ 7 dow temporal cells
<code>entities.parquet</code>	Stage 6	63,614	Per-window entity counts
<code>sentiment.parquet</code>	Stage 7	2	Per-window sentiment (partial run)
<code>topics.parquet</code> / <code>topic_details.parquet</code>	Stage 8	50 / 2	Dominant topic + per-topic words
<code>classifications.parquet</code>	Stage 9	500	5-class predictions
<code>sustain_predictions.parquet</code> / <code>sustain_feature_importance.parquet</code>	Stage 10	500 / 11	Binary predictions + cuML RF importance
<code>forecast_results.parquet</code> / <code>forecast_feature_importance.parquet</code> / <code>forecast_fold_metrics.parquet</code>	Stage 11	138 / 15 / 5	Precursor-window forecaster